

Protótipo de Digital Twin com Modelo Longitudinal Simplificado

Estimativa energética em tempo real a partir de um modelo longitudinal simplificado, integrada por barramento MQTT, persistência temporal, Asset Administration Shell (Eclipse BaSyx) e servidor OPC UA, segundo a estratificação da RAMI 4.0.

Sumário

1	Introdução	2
2	Levantamento de Requisitos	3
2.1	Requisitos Funcionais	3
2.2	Requisitos Não Funcionais	4
2.3	Estado Atual do Projeto	6
2.4	Evolução Implementada	6
2.5	Estrutura de Submodelos AAS (conforme IDTA)	11
2.6	Aquisição de Variáveis do CARLA	14
3	Fundamentação Teórica	16
3.1	Digital Twin	16
3.2	RAMI 4.0	16
3.3	Modelo Longitudinal Simplificado	17
3.4	Convenção de Sinal e Regime Regenerativo	17
3.5	Parâmetros do Modelo	17
4	Metodologia	19
5	Arquitetura do Sistema	20
5.1	Visão Geral Arquitetural	20
5.2	Diagrama de Componentes	20
5.3	Responsabilidades por Camada	20
5.3.1	Camada de Integração (CARLA)	20
5.3.2	Camada Funcional – Domínio	21
5.3.3	Camada Funcional – Aplicação	21
5.3.4	Camada de Informação	21
5.3.5	Camada de Comunicação	21
5.4	Fluxo de Execução	21
5.5	Modos de Operação (Fonte Plugável)	21
5.6	Justificativa Arquitetural	22
6	Resultados Esperados	23
7	Validação do Modelo	24
7.1	Metodologia de Verificação	24
7.2	Resultados Quantitativos	24
7.3	Análise Gráfica	24
7.4	Classificação da Verificação	25
8	Conclusão	26

1

Introdução

Simuladores veiculares como o CARLA fornecem variáveis dinâmicas do veículo, porém não integram explicitamente um modelo energético para estimativa de consumo em tempo real.

Este trabalho propõe o desenvolvimento de um protótipo de Digital Twin energético baseado em um modelo longitudinal simplificado para estimativa de potência instantânea e energia acumulada. O modelo é integrado a um barramento de telemetria assíncrono (MQTT), persistido em banco temporal (TimescaleDB), sincronizado a uma Asset Administration Shell (AAS) via Eclipse BaSyx e exposto industrialmente por meio de um servidor OPC UA, seguindo a estratificação da RAMI 4.0. Uma camada de visualização web (dashboard React, REST + WebSocket) permite acompanhamento em tempo real.

2

Levantamento de Requisitos

Este capítulo apresenta os requisitos funcionais e não funcionais do protótipo de Digital Twin energético.

O sistema possui implementação completa do modelo longitudinal, modelo simplificado de bateria, coordenação energética com indicadores estratégicos, integração com o simulador CARLA, barramento MQTT desacoplado, persistência temporal, sincronização com Asset Administration Shell (AAS) via Eclipse BaSyx (REST + MQTT) e servidor OPC UA para integração industrial, atendendo aos requisitos RF1–RF8.

2.1 Requisitos Funcionais

RF1 – Cálculo de Forças Longitudinais

O sistema deve calcular a força longitudinal equivalente considerando componente inercial, resistência ao rolamento e arrasto aerodinâmico.

RF2 – Cálculo de Potência Mecânica

O sistema deve calcular a potência mecânica instantânea a partir da força longitudinal e da velocidade do veículo.

RF3 – Integração Discreta de Energia

O sistema deve integrar numericamente a energia mecânica acumulada utilizando aproximação discreta baseada no passo de tempo Δt .

RF4 – Atualização do Estado de Carga (SoC)

O sistema deve atualizar o estado de carga da bateria com base na energia elétrica consumida ou regenerada, considerando eficiências simplificadas de descarga e regeneração.

RF5 – Coordenação Energética Unificada

O sistema deve disponibilizar um método unificado `update(request)` por meio da classe `VehicleEnergySystem`, retornando:

- Potência instantânea
- Energia mecânica acumulada

- Estado de carga (SoC)
- Energia elétrica utilizada ou regenerada
- Distância acumulada
- Potência média móvel
- Consumo específico
- Autonomia estimada

RF6 – Integração com Simulador

O sistema deve ser capaz de receber como entrada:

- Velocidade longitudinal
- Aceleração longitudinal
- Intervalo de tempo

A integração direta com a API do CARLA é realizada na camada de Integração, mantendo o domínio desacoplado. A aquisição publica as leituras brutas no barramento MQTT, separando fisicamente aquisição e processamento.

RF7 – Integração com Representação Digital Twin (AAS)

O sistema deve permitir integração com uma instância de Asset Administration Shell (AAS), implementada via Eclipse BaSyx, representando formalmente o Digital Twin energético.

RF8 – Comunicação via OPC UA

O sistema deve expor variáveis energéticas por meio do protocolo OPC UA, possibilitando integração com infraestrutura industrial ou middleware externo (e.g., UAExpert).

A mediação entre o modelo energético e as infraestruturas MQTT/AAS e OPC UA é realizada pela camada de aplicação (serviço de processamento, **TwinService** e o serviço OPC UA), pertencentes às camadas de Informação e Comunicação, respectivamente.

2.2 Requisitos Não Funcionais

RNF1 – Independência do Domínio

A camada de domínio não deve possuir dependência direta do simulador CARLA, BaSyx, OPC UA ou bibliotecas externas de infraestrutura.

RNF2 – Testabilidade Isolada

O modelo energético deve ser executável de forma independente, permitindo validação offline sem necessidade de simulação ou middleware externo.

RNF3 – Separação de Responsabilidades (RAMI 4.0)

A arquitetura segue a estratificação RAMI 4.0, com separação explícita por meio de portas (interfaces) e adaptadores:

- **Ativo:** veículo no simulador CARLA
- **Integração:** aquisição do veículo – `src/infra/carla_client.py`, `src/infra/sources.py`, `src/application/acquisition_service.py`
- **Funcional:** modelo energético e diagnóstico (`src/domain/`) e orquestração/processamento (`src/application/processing_service.py`)
- **Informação:** barramento MQTT (`src/infra/mqtt_client.py`), persistência temporal (`src/infra/timescale_repository.py`) e AAS/BaSyx (`src/application/twin_service.py`, `src/application/twin_sync_service.py`)
- **Comunicação:** servidor OPC UA (`src/application/opcua_service.py`), enlace LoRa (`src/application/lora_service.py`) e API REST + WebSocket (`src/api/server.py`)

RNF4 – Execução Compatível com Passo Discreto

O cálculo energético deve ser executado em tempo compatível com o passo discreto da simulação, garantindo sincronização consistente.

RNF5 – Configurabilidade sem Hardcoding

Todos os parâmetros operacionais (broker, banco, endpoints OPC UA, parâmetros do veículo, limiares de alerta) devem ser definidos por variáveis de ambiente, sem valores fixos no código, com seleção de ambiente (dev/staging/prod). Parâmetros do veículo e limiares de alerta são, ainda, editáveis em tempo de execução pela dashboard.

RNF6 – Extensibilidade Arquitetural

O sistema deve permitir evolução futura para:

- Inclusão de inclinação da via
- Modelos avançados de bateria
- Estratégias energéticas mais complexas
- Autenticação/segurança em OPC UA
- Integração canônica OPC UA → AAS via BaSyx DataBridge
- Representação formal completa em AAS

2.3 Estado Atual do Projeto

O projeto encontra-se na seguinte fase:

- Modelo longitudinal implementado e validado
- Modelo simplificado de bateria implementado e validado
- Sistema coordenador energético com indicadores estratégicos implementado
- Integração com CARLA implementada, com câmera de perseguição e controle manual por teclado (WASD)
- Barramento MQTT desacoplando aquisição, processamento e exposição
- Persistência temporal em PostgreSQL/TimescaleDB, organizada por sessão de teste
- API REST + WebSocket e dashboard web (React) com telemetria ao vivo, pista do trajeto e KPIs
- Integração com AAS via Eclipse BaSyx implementada (REST + MQTT eventing)
- Servidor OPC UA implementado (`asyncua`), parametrizado por ambiente, endpoint padrão `opc.tcp://<host>:4840/marchforce/`
- Enlace LoRa para telemetria resumida ao box (consumidor do barramento)
- Módulo de diagnóstico com códigos de erro e alertas operacionais (enums de severidade/código)
- Sistema de logging estruturado implementado
- 89 testes unitários (padrão Arrange-Act-Assert) cobrindo domínio, diagnóstico, serviços (com fakes das portas), config e exceções
- Simulação sintética offline funcional
- Empacotamento em containers (Docker Compose) com imagens enxutas e testes como gate de build
- Estrutura arquitetural alinhada à RAMI 4.0

Toda a infraestrutura sobe com um único comando (`make run-carla`), que inicia o CARLA, o gêmeo digital (BaSyx) e o *stack* de telemetria, expondo os *endpoints* no IP real do host (Figura 2.1).

A operação do protótipo é padronizada por alvos de `make` (Tabela 2.1), garantindo reprodutibilidade: o mesmo comando builda as imagens enxutas, executa os testes como *gate* de *build* e sobe o *stack*.

2.4 Evolução Implementada

Desde a versão inicial do documento, as seguintes evoluções foram implementadas:

```

Ubuntu
✓Container mosquitto      Removed      0.5s
✓Container mongo          Removed      0.5s
✓Network basyx-setup_default Removed      0.2s
~/interactive-energy-digital-twin-carla develop !16 758 > make run-carla 17s 20:24:31
APP_ENV=dev bash scripts/run_carla.sh
[carla] launching CARLA on the Windows host
Starting CARLA: C:\CARLA\CarlaUE4.exe -carla-rpc-port=2000 -quality-level=Low
CARLA started (PID 24376) on port 2000. Stop it with: Stop-Process -Id 24376
[carla] waiting for CARLA at 10.255.255.254:2000
[carla] timeout waiting for CARLA - continuing anyway
[carla] bringing up the stack in CARLA mode (no in-container acquisition)
[deploy] bringing up the digital twin (BaSyx: AAS + Mongo + MQTT/OPC UA)
[+] Running 1/7
✓Network basyx-setup_default Created      0.1s
✓Container mosquitto      Healthy    13.1s
✓Container mongo          Healthy    13.1s
✓Container aas-registry    Healthy    58.9s
✓Container dashboard-api   Started    12.2s
✓Container sm-registry     Healthy    58.9s
✓Container aas-discovery   Started    12.3s
✓Container aas-env         Healthy    185.0s
✓Container aas-ui          Started    185.4s
[deploy] starting stack (APP_ENV=dev, source=carla, twin=1)
[+] Running 7/7
✓Network marchforce-telemetry_default Created      0.1s
✓Container mf-timescaledb   Healthy     6.9s
✓Container mf-opcua         Started     7.3s
✓Container mf-processing    Started     7.2s
✓Container mf-twin-sync     Started     7.3s
✓Container mf-api           Started     7.2s
✓Container mf-dashboard     Started     7.6s
[deploy] dashboard: http://localhost:8080 - API: http://localhost:8080
[deploy] OPC UA (communication layer): opc.tcp://172.28.15.103:4840
[deploy] BaSyx AAS (digital twin): http://localhost:8081 - AAS UI: http://localhost:3080
[carla] poetry not found - using .venv (the carla wheel may be missing)
[carla] starting host acquisition (CARLA -> MQTT) via: .venv/bin/python
[carla] acquisition started (pid 248892). Dashboard: http://localhost:8080
~/interactive-energy-digital-twin-carla develop !16 758 >
6m 12s 20:30:50

```

Figura 2.1: Implantação por um comando: o CARLA, o gêmeo digital BaSyx (9 contêineres) e o *stack* de telemetria (7 contêineres) sobem saudáveis, com os *endpoints* publicados no IP do host.

Sistema de Logging Estruturado

Foi implementado um sistema centralizado de logging utilizando a biblioteca `structlog`, configurado em `src/infra/logger.py`. A camada de domínio permanece livre de dependências de logging, conforme RNF1.

Barramento MQTT e Separação Aquisição/Processamento

A aquisição (CARLA) publica leituras brutas no tópico `marchforce/telemetry/raw`. O serviço de processamento (`src/application/processing_service.py`) consome esse tópico, executa o modelo energético, persiste a série temporal e republica o estado processado em `marchforce/telemetry/processed`. O cliente MQTT (`src/infra/mqtt_client.py`) é o barramento lógico que desacopla os serviços; produtores e consumidores nunca se comunicam diretamente.

Persistência Temporal (TimescaleDB)

O repositório `src/infra/timescale_repository.py` armazena a telemetria processada em PostgreSQL/TimescaleDB (hypertable), organizada por sessão de teste, permitindo consulta de histórico e comparação entre execuções via API REST.

Exposição via API REST + WebSocket e Dashboard Web

A camada de exposição (`src/api/server.py`, FastAPI) entrega o histórico por REST e retransmite a telemetria processada por WebSocket. A dashboard (React + Vite) apresenta KPIs, gráficos em tempo real, painel de alertas, a pista do trajeto (coordenadas reais do

Tabela 2.1: Principais comandos de operação (**make**).

Comando	Descrição
<code>make run</code>	Builda as imagens (testes como <i>gate</i>) e sobe o <i>stack</i>
<code>make run-carla</code>	Inicia o CARLA e sobe todo o <i>stack</i> em modo CARLA
<code>make verify</code>	<i>Smoke test</i> do <i>stack</i> no ar (API, OPC UA, twin-sync)
<code>make test</code>	Suíte unitária (<i>pytest</i> , padrão AAA)
<code>make smoke</code>	Pipeline ponta a ponta offline (<i>fakes</i> , sem infraestrutura)
<code>make dev</code>	Execução de desenvolvimento com <i>hot reload</i>
<code>make down</code>	Derruba o <i>stack</i>
<code>make report</code>	Compila este relatório em PDF

CARLA) e modais de edição de parâmetros e limiares de alerta. Substitui a visualização HUD anterior.

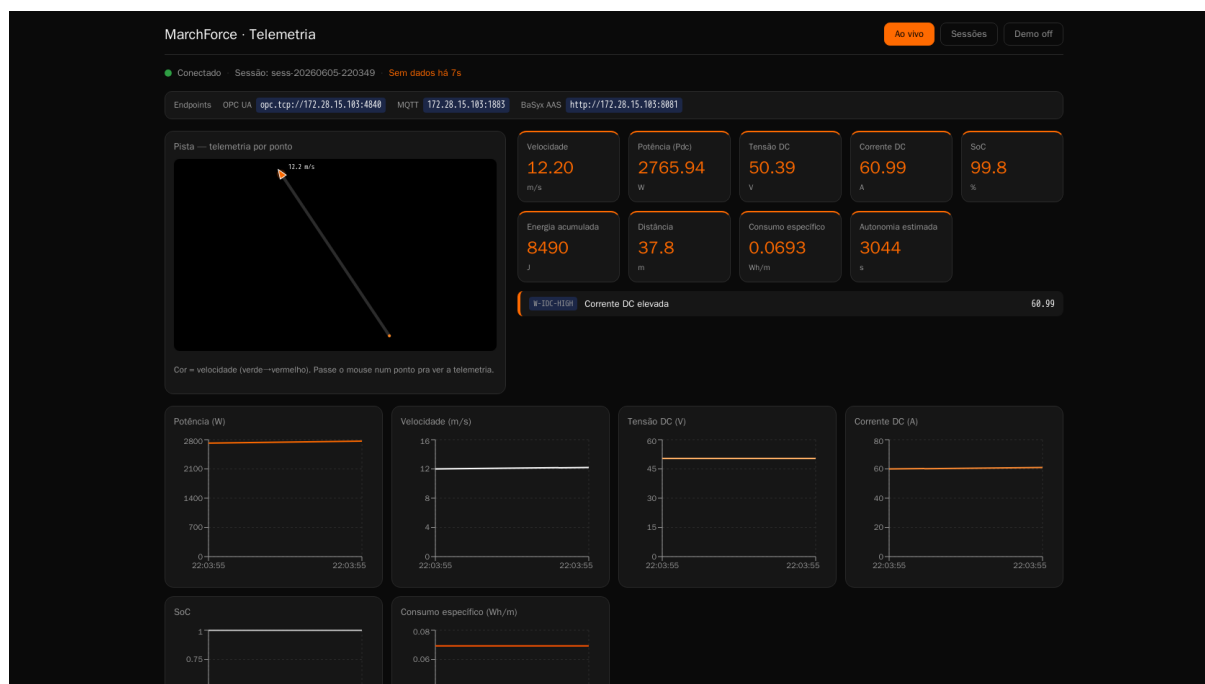


Figura 2.2: Dashboard web em tempo real (React + Vite): *endpoints* no IP real do host, pista do trajeto, cartões de KPI, gráficos ao vivo e painel de alertas (no exemplo, o alerta W-IDC-HIGH de corrente DC elevada).

Integração com Asset Administration Shell (BaSyx)

A classe `TwinService` (camada de Informação) realiza a sincronização com a AAS `vehicle-digitaltwin.aasx` via Eclipse BaSyx v2 (*aas-environment*, API REST). Consumindo o tópico `marchforce/telemetry/processed`, o serviço atualiza em tempo real os KPIs do submodelo **EnergyEfficiency** (*Distance*, *AveragePower*, *SpecificConsumption* e *Autonomy*) por PATCH REST, de forma *orientada ao modelo canônico* (`aas_model.TEL`

EMETRY_MODEL). A série temporal crua não é replicada na AAS: ela é referenciada pelo `LinkedSegment` do submodelo `TimeSeries` (apontando para o TimescaleDB via API), conforme o SMT IDTA 02008. Assim, mantém-se uma única fonte de dados (sem segunda execução do CARLA) e a AAS permanece uma representação dinâmica do ativo.

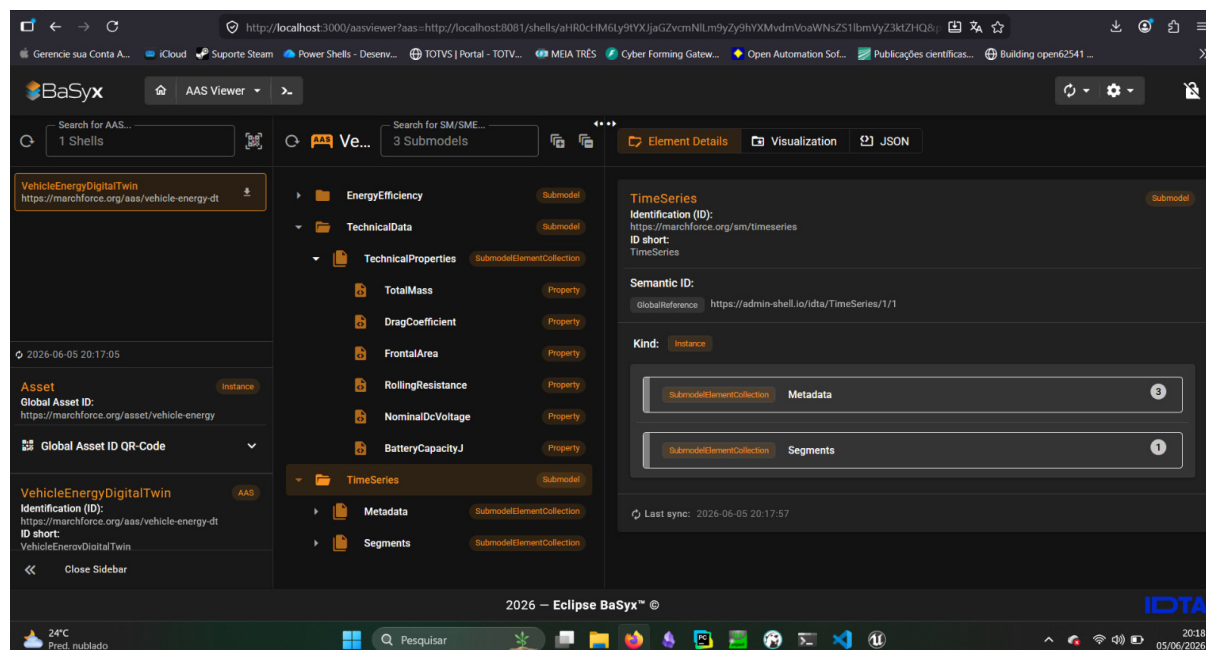


Figura 2.3: Asset Administration Shell carregada no Eclipse BaSyx Web UI: a *shell* `VehicleEnergyDigitalTwin` e os três submodelos IDTA (`EnergyEfficiency`, `TechnicalData` e `TimeSeries`).

A AAS efetivamente carregada e sincronizada foi verificada pela API REST do BaSyx (Listagem 2.4): a *shell* e os três submodelos IDTA aparecem, e o submodelo `EnergyEfficiency` exibe os KPIs já atualizados pelo *twin-sync*.

```
GET /shells      -> ["VehicleEnergyDigitalTwin"]
GET /submodels  -> ["TimeSeries", "EnergyEfficiency", "TechnicalData"]
GET /submodels/<EnergyEfficiency>/submodel-elements
  Distance      = 358.5
  AveragePower  = 1665.45
  SpecificConsumption = 0.0494
  Autonomy      = 2963.9
```

Figura 2.4: Evidência REST: AAS carregada no BaSyx v2 e KPIs de `EnergyEfficiency` atualizados pelo *twin-sync*.

A AAS também pode ser inspecionada pelo BaSyx Web UI (Figura 2.3) e pela API REST documentada em *Swagger* (Figura 2.5).

Integração OPC UA (RF8: Camada de Comunicação)

Foi implementado o servidor OPC UA em `src/application/opcu_service.py` com a biblioteca `asyncua`. O servidor é *totalmente parametrizado por ambiente* (host, porta,

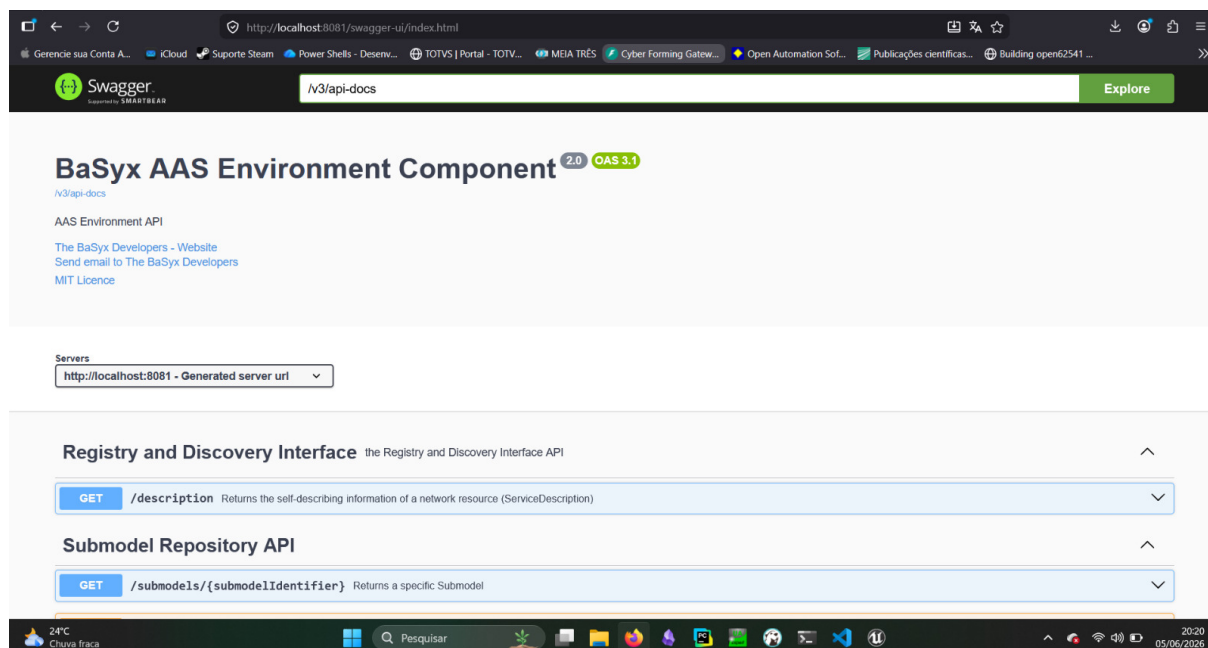


Figura 2.5: API REST do BaSyx v2 (*aas-environment*, OpenAPI 3.1) usada pelo *twin-sync* para sincronizar os submodelos.

path, *namespace*, nome do servidor e intervalo de atualização), sem valores fixos no código (RNF5). Ele consome o tópico `marchforce/telemetry/processed` e expõe a *shell* do gêmeo digital, cujos nós são atualizados periodicamente.

Um ponto central de projeto é que a *estrutura de nós OPC UA é construída a partir do modelo canônico* (`src/domain/aas_model.py`, `TELEMETRY_MODEL`), o mesmo que origina o Record da AAS, **espelhando a hierarquia do AAS V3**: o objeto raiz recebe o *idShort* da *shell* (`VehicleEnergyDigitalTwin`) e dela pendem, como filhos diretos, os três submodelos. Assim, em vez de uma lista plana de variáveis, os nós ficam *agrupados por submodelo*: `TimeSeries` (com `Velocity`, `Acceleration`, `InstantaneousPower`, `AccumulatedEnergy`, `DcVoltage`, `DcCurrent`, `StateOfCharge`), `EnergyEfficiency` (com `Distance`, `AveragePower`, `SpecificConsumption`, `Autonomy`) e `TechnicalData` (parâmetros estáticos do veículo). Os *idShort* OPC UA coincidem com os da AAS, garantindo uma única fonte estrutural: alterar o modelo canônico reflete simultaneamente na AAS e no espaço de endereçamento OPC UA. Endpoint padrão: `opc.tcp://<host>:4840/marchforce/`, conectável por clientes como o UAExpert.

Cabe registrar a fronteira RAMI 4.0 adotada: o servidor *asynqua* provê a *camada de Comunicação* (protocolo OPC UA), enquanto o BaSyx serve a AAS pela API REST (camada de Informação). A ponte canônica OPC UA → AAS via BaSyx *DataBridge* é trabalho futuro (Seção de limitações).

O espaço de endereçamento foi inspecionado por um cliente OPC UA (Listagem 2.7), confirmando os nós agrupados por submodelo e os valores atualizando a partir do barramento MQTT.

O servidor é localizável na rede tanto em `localhost` quanto no IP real do host (Figura 2.8), de modo que clientes externos (e.g., o *box* ou uma ferramenta de engenharia) o descubrem e conectam, aqui com autenticação anônima.

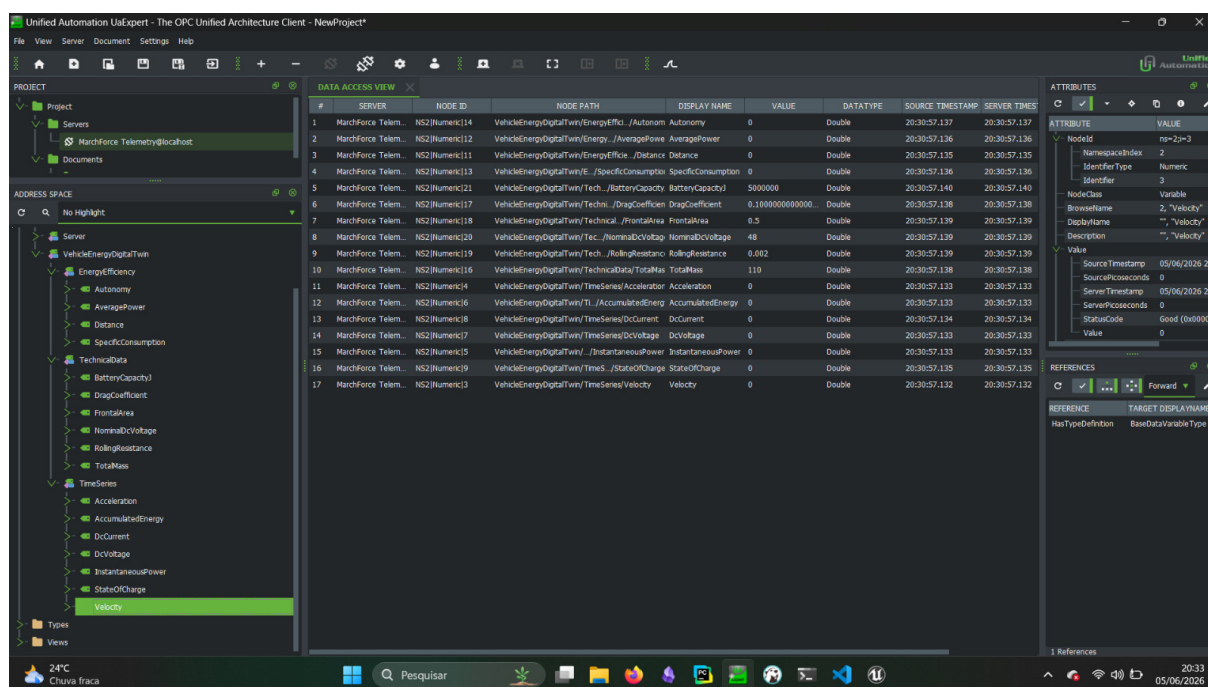


Figura 2.6: Servidor OPC UA inspecionado no UAExpert: a *shell* VehicleEnergyDigitalTwin como raiz, os submodelos EnergyEfficiency, TechnicalData e TimeSeries como filhos diretos, e os valores atualizando em tempo real.

Enlace LoRa (Telemetria Externa)

O serviço `src/application/lora_service.py` consome o barramento processado e transmite um subconjunto de variáveis essenciais e alertas críticos ao box, respeitando MTU pequeno e duty cycle. O enlace é um *port* TelemetryLink, com adaptadores de *log* (sem hardware) e serial.

Diagnóstico e Alertas Operacionais

O módulo `src/domain/diagnostics.py` avalia cada amostra e emite alertas com código (AlertCode), severidade (Severity) e mensagem, propagados no barramento e exibidos na dashboard, cobrindo SoC, tensão/corrente DC, velocidade, leituras inválidas e falhas de persistência.

2.5 Estrutura de Submodelos AAS (conforme IDTA)

A modelagem da Asset Administration Shell segue os *Submodel Templates* (SMTs) padronizados da *Industrial Digital Twin Association* (IDTA), garantindo interoperabilidade. A AAS (`aas-model/vehicle-digitaltwin.aasx`) é composta por três submodelos (Tabela 2.2):

- **TimeSeries** (IDTA 02008-1-1 *Time Series Data*, `semanticId` <https://admin-shell.io/idta/TimeSeries/1/1>): descreve a telemetria *dinâmica*. O Metadata/Record define o *schema* das variáveis e o Segments/LinkedSegment referencia a base temporal (TimescaleDB) via Endpoint/Query da API, sem duplicar os dados na AAS.

```
opc.tcp://172.28.15.103:4840/marchforce/
'-- VehicleEnergyDigitalTwin      (shell / AAS)
  |-- TimeSeries
  |   |-- Velocity                = 30.00
  |   |-- Acceleration            = 0.00
  |   |-- InstantaneousPower      = 891.62
  |   |-- AccumulatedEnergy       = 69047.53
  |   |-- DcVoltage               = 50.33
  |   |-- DcCurrent               = 19.69
  |   |-- StateOfCharge           = 98.47
  |-- EnergyEfficiency
  |   |-- Distance                = 891.88
  |   |-- AveragePower            = 891.62
  |   |-- SpecificConsumption    = 0.023894
  |   |-- Autonomy                = 5521.7
  '-- TechnicalData
      |-- TotalMass               = 110.0
      |-- DragCoefficient         = 0.10
      |-- FrontalArea             = 0.5
      |-- RollingResistance       = 0.002
      |-- NominalDcVoltage        = 48.0
      '-- BatteryCapacityJ        = 5000000.0
```

Figura 2.7: Árvore de nós OPC UA lida por cliente (*asyncua*): a *shell* *VehicleEnergyDigitalTwin* como raiz e os submodelos *TimeSeries*, *EnergyEfficiency* e *TechnicalData* como filhos diretos (AAS V3).

- **EnergyEfficiency:** KPIs *derivados* de gestão de energia, atualizados em tempo real na AAS via PATCH REST.
- **TechnicalData** (IDTA 02003): parâmetros *estáticos* do veículo.

A série temporal crua é exposta pelo *LinkedSegment* (apontando para o TimescaleDB via API), enquanto os KPIs de eficiência são atualizados na AAS via PATCH REST do BaSyx v2 (`/submodels/{base64(submodelId)}/submodel-elements/{idShort}/${value}`), alimentados pelo consumo do tópico `marchforce/telemetry/processed`. Os identificadores seguem codificação Base64 URL-safe conforme AAS Part 2. O arquivo `.aasx` foi validado com o `basyx-python-sdk`.

Indicadores Estratégicos de Eficiência

A classe *VehicleEnergySystem* calcula:

- **Distância acumulada:**

$$d(t) = \sum_{k=0}^n |v_k| \cdot \Delta t \quad (2.1)$$

- **Potência média móvel** (janela de 50 passos):

$$\bar{P}(t) = \frac{1}{N} \sum_{i=t-N}^t P(i) \quad (2.2)$$

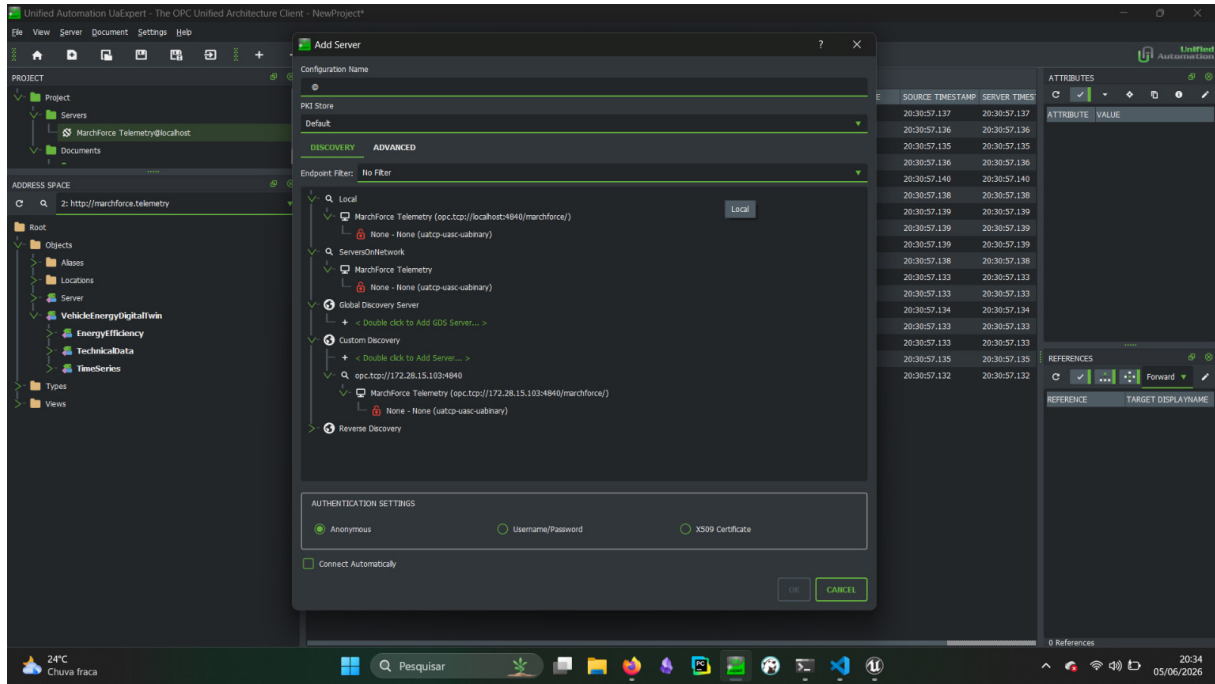


Figura 2.8: Descoberta do servidor OPC UA no UAExpert: o *endpoint* MarchForce Telemetry é encontrado em `opc.tcp://localhost:4840/marchforce/` e no IP real do host (`opc.tcp://172.28.15.103:4840/marchforce/`).

- **Consumo específico:**

$$C_{esp} = \frac{E_{elec}/3600}{d} \quad [\text{Wh/m}] \quad (2.3)$$

- **Autonomia estimada:**

$$A(t) = \frac{SoC \cdot C_{bat}}{\bar{P}(t)} \quad [\text{s}] \quad (2.4)$$

A estimativa é definida apenas quando $\bar{P}(t)$ supera um piso mínimo, evitando divisão por valores próximos de zero (carro parado/cruzeiro).

Validação por Testes Unitários

Foram implementados 89 testes unitários, no padrão Arrange–Act–Assert, cobrindo:

- **EnergyModel:** cálculo de forças, potência, acumulação de energia
- **Battery:** descarga, regeneração, limitação do SoC
- **VehicleEnergySystem:** indicadores estratégicos, distância, média móvel
- **Diagnostics:** enums, limiares e geração de alertas
- **Serviços** (aquisição, processamento, LoRa, twin-sync) com *fakes* que implementam as portas, sem broker ou banco
- **Configuração,** fontes, portas e hierarquia de exceções

A validação offline confirma RNF2.

Tabela 2.2: Submodelos AAS (IDTA) e variáveis

Submodelo (SMT)	Elemento (idShort)	Variável	Natureza
TimeSeries (02008)	Record.Velocity	v_{long}	dinâmica (DB)
TimeSeries (02008)	Record.Acceleration	a_{long}	dinâmica (DB)
TimeSeries (02008)	Record.InstantaneousPower	P	dinâmica (DB)
TimeSeries (02008)	Record.AccumulatedEnergy	E	dinâmica (DB)
TimeSeries (02008)	Record.DcVoltage	V_{dc}	dinâmica (DB)
TimeSeries (02008)	Record.DcCurrent	I_{dc}	dinâmica (DB)
TimeSeries (02008)	Record.StateOfCharge	SoC	dinâmica (DB)
EnergyEfficiency	Distance	d	dinâmica (PATCH)
EnergyEfficiency	AveragePower	$\bar{P}$	dinâmica (PATCH)
EnergyEfficiency	SpecificConsumption	C_{esp}	dinâmica (PATCH)
EnergyEfficiency	Autonomy	A	dinâmica (PATCH)
TechnicalData (02003)	TotalMass	m	estática
TechnicalData (02003)	DragCoefficient	C_d	estática
TechnicalData (02003)	FrontalArea	A_f	estática
TechnicalData (02003)	RollingResistance	C_r	estática
TechnicalData (02003)	NominalDcVoltage	V_{nom}	estática
TechnicalData (02003)	BatteryCapacityJ	C_{bat}	estática

Simulação Sintética

O script `scripts/simulate.py` executa o pipeline energético completo sem dependência de CARLA/BaSyx, com três fases: aceleração (0–15 s), cruzeiro (15–40 s) e frenagem regenerativa (40–60 s). Um *smoke test* (`scripts/smoke_pipeline.py`) valida o caminho ponta a ponta (fonte → processamento → consumidores) sem infraestrutura externa.

2.6 Aquisição de Variáveis do CARLA

O simulador fornece velocidade como vetor 3D. Para a componente longitudinal, projeta-se no *forward vector*:

$$v_{long} = \vec{v} \cdot \hat{f} = v_x f_x + v_y f_y + v_z f_z \quad (2.5)$$

Onde $\hat{f}$ é obtido via `vehicle.get_transform().get_forward_vector()`.

A aceleração longitudinal é derivada da velocidade entre passos consecutivos, filtrada por média móvel exponencial e limitada a uma faixa física, evitando picos não-físicos da leitura bruta do simulador (colisões/ruído):

$$a_{long}(t) = \text{clamp}\left(\frac{v_{long}(t) - v_{long}(t - \Delta t)}{\Delta t}, -a_{max}, a_{max}\right) \quad (2.6)$$

O veículo é conduzido no simulador por teclado (WASD, com S acionando a ré) e acompanhado por uma câmera de perseguição. A aquisição roda no host (que possui a *wheel* do CARLA) e publica as leituras no barramento MQTT, de modo que a dashboard desenha o trajeto e exibe a telemetria energética em tempo real, sem nenhum acoplamento entre o domínio e o simulador (Figuras 2.9 e 2.10).

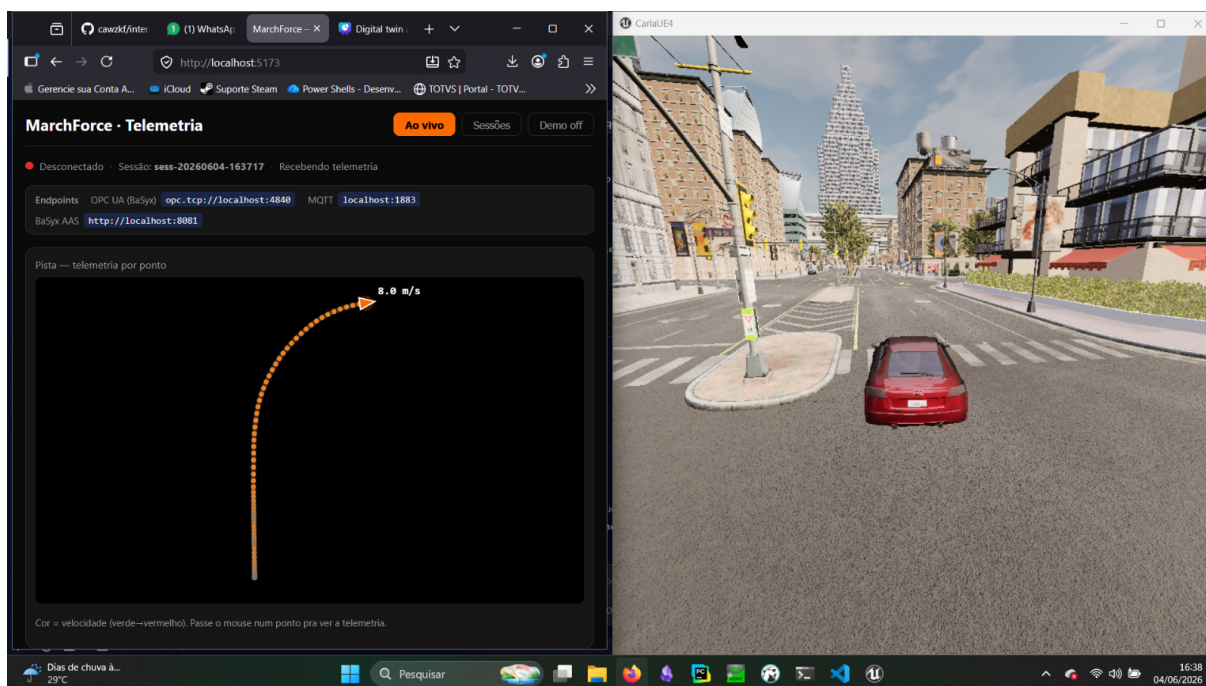


Figura 2.9: Aquisição com CARLA: à direita o veículo é dirigido no simulador (WASD, câmera de perseguição); à esquerda a dashboard recebe a telemetria pelo barramento e desenha o trajeto em tempo real.



Figura 2.10: Veículo no CARLA com câmera de perseguição, controlado por teclado (WASD; S aciona a ré).

3

Fundamentação Teórica

3.1 Digital Twin

O conceito de *Digital Twin* foi formalizado por Grieves [3], sendo expandido como integração dinâmica entre sistemas físicos e modelos computacionais conectados em tempo real [2].

Neste trabalho considera-se apenas dinâmica longitudinal em plano horizontal.

3.2 RAMI 4.0

O *Reference Architectural Model Industrie 4.0* (RAMI 4.0) organiza sistemas ciberfísicos em camadas hierárquicas: Ativo, Integração, Comunicação, Informação, Funcional e Negócio. Este trabalho atende explicitamente as cinco primeiras (Figura 3.1): o OPC UA é o protocolo da camada de Comunicação (servidor *asyncua*), e o MQTT, a persistência temporal e o AAS/BaSyx compõem a camada de Informação. Cabe destacar que, no Eclipse BaSyx v2, o *aas-environment* expõe o AAS por REST; a integração canônica de protocolos de campo (OPC UA, MQTT) ao AAS é feita pelo componente *DataBridge* (cliente), prevista como evolução.

Negócio	<i>fora do escopo deste protótipo</i>
Funcional	Modelo energético, diagnóstico e processamento
Informação	MQTT, TimescaleDB e AAS via BaSyx
Comunicação	Servidor OPC UA, API/WebSocket e LoRa
Integração	Aquisição do CARLA (RS485/B.O.M. no futuro)
Ativo	Veículo (simulado no CARLA)

Figura 3.1: Camadas da RAMI 4.0 e o mapeamento para os componentes do protótipo. As cinco camadas inferiores (Ativo a Funcional) são atendidas; a camada de Negócio está fora do escopo.

3.3 Modelo Longitudinal Simplificado

Modelos longitudinais são discutidos na literatura de dinâmica veicular [5]:

$$F_{rol} = C_r m g \quad (3.1)$$

$$F_{aero} = \frac{1}{2} \rho C_d A v^2 \quad (3.2)$$

$$F_{long} = m \cdot a + F_{rol} + F_{aero} \quad (3.3)$$

Onde m é a massa [kg], a a aceleração [m/s²], C_r o coeficiente de rolamento, g a gravidade [m/s²], ρ a densidade do ar [kg/m³], C_d o coeficiente aerodinâmico, A a área frontal [m²] e v a velocidade [m/s].

A modelagem de bateria segue abordagem energética de alto nível [4].

$$P = F_{long} \cdot v \quad (3.4)$$

$$E = \int P dt \quad (3.5)$$

$$E_k = E_{k-1} + P_k \cdot \Delta t \quad (3.6)$$

3.4 Convenção de Sinal e Regime Regenerativo

- $P > 0$: tração – energia da bateria
- $P = 0$: equilíbrio
- $P < 0$: frenagem regenerativa – energia à bateria

$$E_{elec} = \frac{P \cdot \Delta t}{\eta_d} \quad (3.7)$$

$$E_{regen} = |P| \cdot \Delta t \cdot \eta_r \quad (3.8)$$

$$SoC_{k+1} = SoC_k - \frac{E_{elec}}{C_{bat}} \quad (\text{tração}) \quad (3.9)$$

$$SoC_{k+1} = \min \left(1, SoC_k + \frac{E_{regen}}{C_{bat}} \right) \quad (\text{regeneração}) \quad (3.10)$$

3.5 Parâmetros do Modelo

Os parâmetros são configuráveis por ambiente (RNF5). A Tabela 3.1 apresenta a configuração de referência utilizada na verificação analítica (veículo de passeio). O *deploy* de produção adota uma configuração de protótipo leve (e.g., massa total ≈ 110 kg, $C_d \approx 0,10$, $A \approx 0,5$ m², $C_r \approx 0,002$), editável em tempo de execução pela dashboard.

Os parâmetros do veículo e os limiares de alerta são, ainda, editáveis em tempo de execução pela dashboard (Figura 3.2): o modal publica a nova configuração no tópico `mar chforce/config`, que o serviço de processamento consome e aplica sem reiniciar o stack.

Tabela 3.1: Parâmetros de referência do modelo energético (verificação)

Parâmetro	Símbolo	Valor	Unidade
Massa do veículo	m	1500	kg
Coefficiente aerodinâmico	C_d	0,3	—
Área frontal	A	2,2	m ²
Coefficiente de rolamento	C_r	0,015	—
Densidade do ar	ρ	1,225	kg/m ³
Aceleração da gravidade	g	9,81	m/s ²
Capacidade da bateria	C_{bat}	50	MJ
Eficiência de descarga	η_d	0,9	—
Eficiência de regeneração	η_r	0,9	—
Passo de simulação	Δt	0,1	s

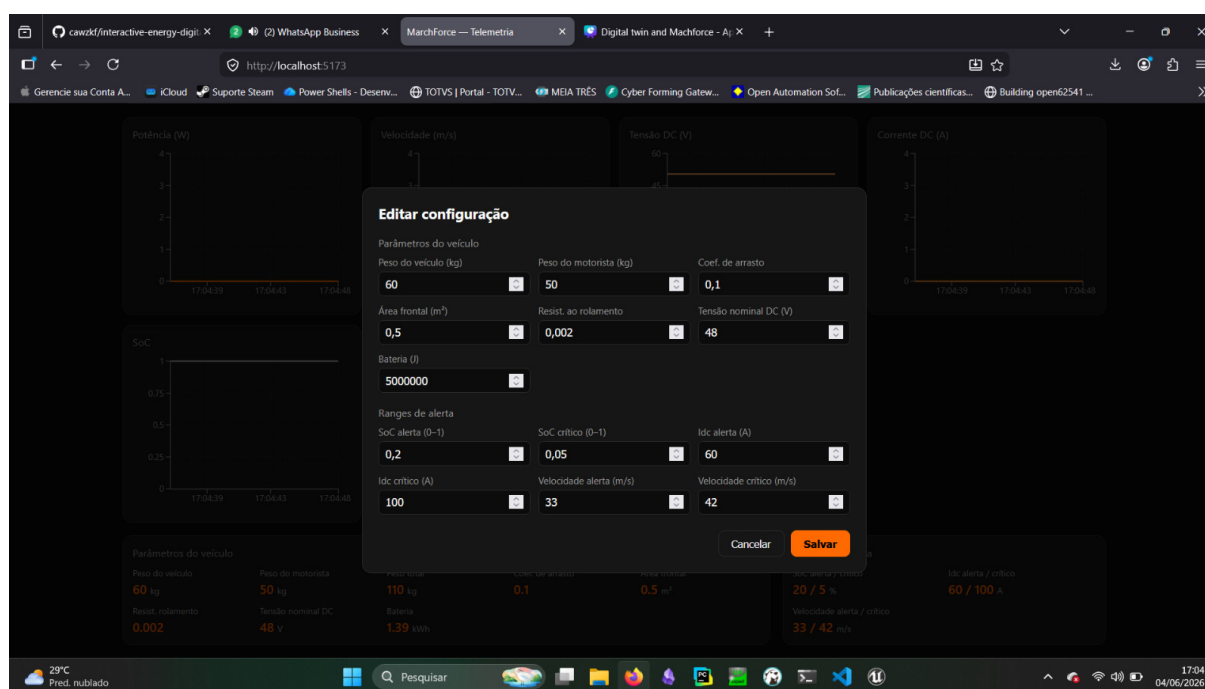


Figura 3.2: Modal de edição de configuração na dashboard: parâmetros do veículo (massas, C_d , área frontal, C_r , tensão nominal, capacidade da bateria) e limiares de alerta (SoC, corrente DC e velocidade).

4

Metodologia

A implementação foi realizada em Python utilizando a API oficial do simulador CARLA [1]. O sistema é estruturado conforme a RAMI 4.0:

- **Integração:** aquisição de dados do CARLA (`src/infra/carla_client.py`, `src/infra/sources.py`) e publicação no MQTT (`src/application/acquisition_service.py`)
- **Funcional:** modelo energético e diagnóstico (`src/domain/`) e processamento (`src/application/processing_service.py`)
- **Informação:** MQTT (`src/infra/mqtt_client.py`), persistência (`src/infra/timescale_repository.py`) e AAS/BaSyx (`src/application/twin_service.py`, `twin_sync_service.py`)
- **Comunicação:** servidor OPC UA (`src/application/opcua_service.py`), enlace LoRa (`src/application/lora_service.py`) e API/WebSocket (`src/api/server.py`)

O isolamento entre camadas é garantido por portas (interfaces abstratas em `src/application/ports.py`) e adaptadores concretos em `src/infra` e `src/application`.

5

Arquitetura do Sistema

5.1 Visão Geral Arquitetural

A arquitetura foi estruturada segundo o princípio de separação de responsabilidades, alinhada à RAMI 4.0. O objetivo principal é garantir desacoplamento entre o modelo energético e a infraestrutura externa (CARLA, MQTT, TimescaleDB, BaSyx, OPC UA, LoRa), mediado por um barramento de mensagens.

5.2 Diagrama de Componentes

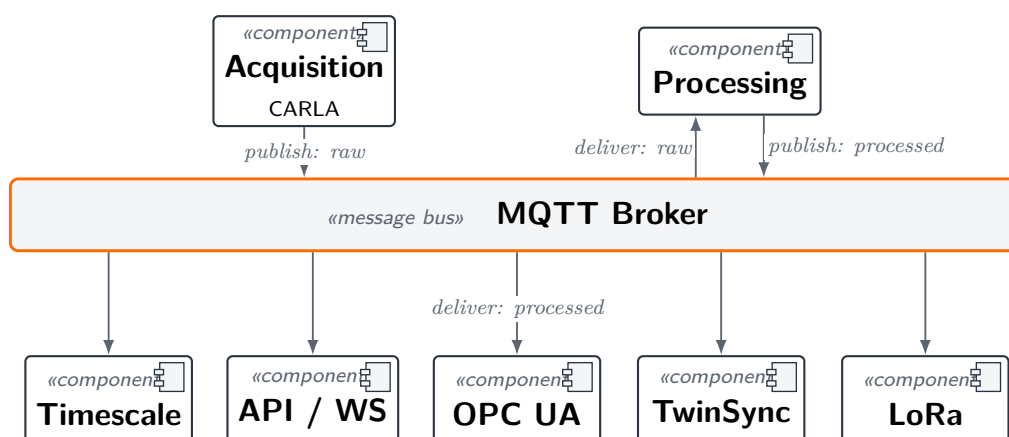


Figura 5.1: Diagrama de componentes (UML): produtores e consumidores acoplados apenas pelo barramento MQTT. Acquisition publica em `telemetry/raw`; Processing consome `raw`, executa o modelo energético e publica em `telemetry/processed`; os demais componentes assinam `processed`.

5.3 Responsabilidades por Camada

Camada de Integração (CARLA)

Obter velocidade (v), aceleração (a) e passo de tempo (dt) e publicar a leitura bruta no MQTT. Não realiza cálculos energéticos. Inclui câmera de perseguição e controle manual por teclado (WASD), com alternância para piloto automático (Traffic Manager).

Camada Funcional – Domínio

Modelo físico-matemático: cálculo de forças, potência, integração discreta de energia, atualização do SoC e diagnóstico. Sem dependência direta do CARLA.

Camada Funcional – Aplicação

Consome o tópico bruto, invoca `update(v,a,dt)`, deriva as grandezas de barramento DC (V_{dc} , I_{dc}), persiste a série temporal e republica o estado processado no MQTT.

Camada de Informação

Barramento MQTT, persistência por sessão em TimescaleDB e sincronização dos submodelos AAS via REST no BaSyx (alimentado pelo `TwinSyncService`).

Camada de Comunicação

Expõe as variáveis energéticas em um servidor OPC UA (`opc.tcp://<host>:4840/marchforce/`), entrega telemetria à dashboard por API REST + WebSocket e envia telemetria resumida ao box por LoRa.

5.4 Fluxo de Execução

O fluxo de telemetria é apresentado como diagrama de sequência UML na Figura 5.2.

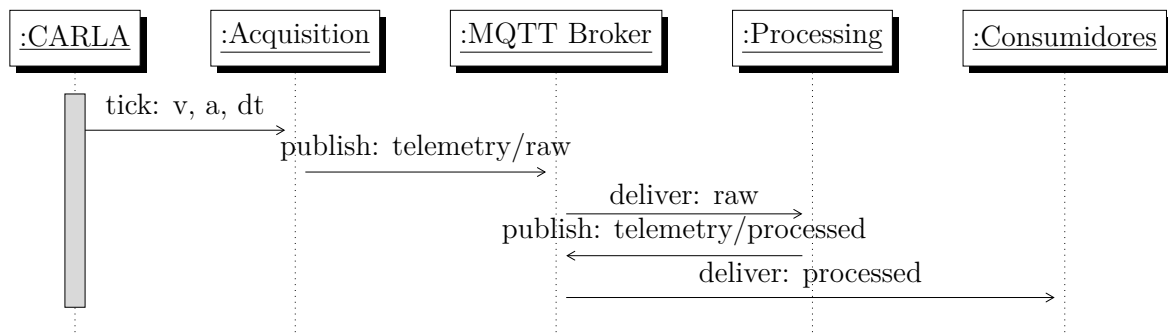


Figura 5.2: Diagrama de sequência (UML) do fluxo de telemetria: a aquisição publica leituras brutas, o processamento executa o modelo e republica o estado processado, e os consumidores (TimescaleDB, API/WebSocket, OPC UA, BaSyx, LoRa) assinam o tópico processado.

5.5 Modos de Operação (Fonte Plugável)

Como todo o pipeline consome exclusivamente do barramento MQTT, a fonte de telemetria é *plugável*: trocá-la não altera processamento, persistência, OPC UA, AAS, LoRa ou dashboard (RNF1). São suportados três modos:

- **CARLA:** a aquisição roda no host (que possui a *wheel* do CARLA), lê o veículo no simulador, controlado por teclado (WASD), e publica em `marchforce/telemetry/raw` (`ACQ_SOURCE=carla`, alvo `make run-carla`).
- **Simulador ou sensores reais:** o simulador embutido publica telemetria sintética no mesmo tópico (`ACQ_SOURCE=sim`, padrão de `make run`); quando o B.O.M. for liberado, os sensores reais publicam a leitura bruta no `raw` pela ponte RS485→MQTT, *sem qualquer alteração no restante do sistema*: produtores e consumidores nunca se conhecem.
- **Demo:** a dashboard gera telemetria sintética no próprio navegador (`useDemoTelemetry`), sem *backend*, MQTT ou CARLA, permitindo demonstrar a interface com o *stack* desligado.

5.6 Justificativa Arquitetural

A estratificação RAMI 4.0, associada ao barramento MQTT e às portas/adaptadores, isola o modelo físico-matemático da infraestrutura de integração e comunicação, permitindo validação independente, testes unitários e substituição de fontes de dados (e.g., hardware real, ponte RS485) sem alteração da lógica do modelo.

6

Resultados Esperados

O protótipo implementado apresenta:

- Estimativa energética coerente, validada por 89 testes unitários e simulação sintética
- Indicadores estratégicos calculados em tempo real
- Barramento MQTT desacoplado e persistência temporal por sessão
- Sincronização com AAS via Eclipse BaSyx (REST + MQTT)
- Exposição das variáveis via OPC UA (parametrizado), cobrindo as camadas de Informação e Comunicação da RAMI 4.0
- Dashboard web (React) com telemetria ao vivo, pista do trajeto, alertas e edição de configuração
- Telemetria resumida ao box via enlace LoRa

7

Validação do Modelo

7.1 Metodologia de Verificação

Verificação por comparação entre o modelo implementado e o cálculo analítico direto, em cenário sintético de três fases, utilizando a configuração de referência (Tabela 3.1):

- 1. **Aceleração** (0–15 s): $a = 2,0 \text{ m/s}^2$, v de 0 a 30 m/s
- 2. **Cruzeiro** (15–40 s): $a = 0$, $v = 30 \text{ m/s}$
- 3. **Frenagem regenerativa** (40–60 s): $a = -1,5 \text{ m/s}^2$, v de 30 a 0 m/s

7.2 Resultados Quantitativos

Tabela 7.1: Verificação de energia acumulada por fase (configuração de referência)

Fase	E_{modelo} (kJ)	$E_{analitico}$ (kJ)	Erro (%)
Aceleração (0–15 s)	760,22	760,22	0,0000
Cruzeiro (15–40 s)	447,41	447,41	0,0000
Frenagem (40–60 s)	0,00	0,00	0,0000
Total	1207,63	1207,63	0,0000

O erro nulo confirma que a implementação reproduz fielmente a formulação matemática.

7.3 Análise Gráfica

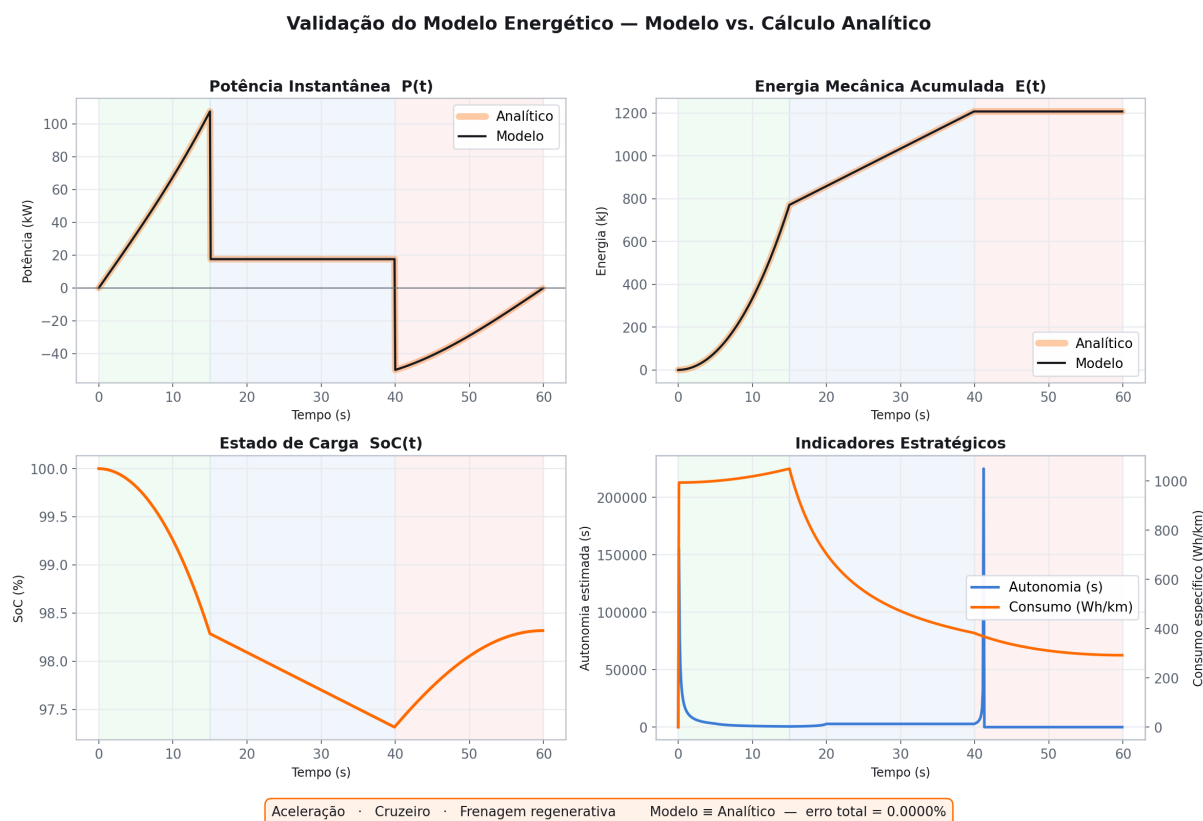


Figura 7.1: Validação: (a) potência instantânea, (b) energia mecânica, (c) SoC, (d) indicadores estratégicos.

- **$P(t)$** : modelo e analítico sobrepostos; cruzeiro em $\approx 17,5$ kW; frenagem com pico de ≈ -50 kW
- **$E(t)$** : crescente em tração, estável em frenagem
- **SoC(t)**: 100% $\rightarrow$ 97,3% em tração; recupera para 98,3% com regeneração
- **Indicadores**: autonomia ≈ 2800 s em cruzeiro; consumo específico ≈ 380 Wh/km

7.4 Classificação da Verificação

Esta validação caracteriza uma *verificação de implementação (code verification)*. A *validação experimental* contra dados reais é prevista como trabalho futuro.

8

Conclusão

O protótipo desenvolvido integra modelagem física simplificada com ambiente de simulação realista (CARLA), barramento de telemetria desacoplado (MQTT), persistência temporal (TimescaleDB), representação formal via Asset Administration Shell (Eclipse BaSyx), exposição industrial via OPC UA e visualização web em tempo real, cobrindo as camadas de Integração, Funcional, Informação e Comunicação da RAMI 4.0. Os 89 testes unitários e a verificação analítica confirmam a correção numérica do modelo, enquanto a arquitetura desacoplada por portas e adaptadores — com configuração integral por ambiente, sem *hardcoding* — permite evolução futura para sistemas embarcados, integração com hardware real e a ponte canônica OPC UA → AAS via BaSyx DataBridge.

Bibliografia

- [1] Alexey Dosovitskiy et al. “CARLA: An Open Urban Driving Simulator”. Em: *Conference on Robot Learning (CoRL)*. 2017, pp. 1–16.
- [2] Aidan Fuller et al. “Digital Twin: Enabling Technologies, Challenges and Open Research”. Em: *IEEE Access* 8 (2020), pp. 108952–108971. DOI: 10.1109/ACCESS.2020.2998358.
- [3] Michael Grieves. *Digital Twin: Manufacturing Excellence through Virtual Factory Replication*. White paper. 2014.
- [4] James Larminie e John Lowry. *Electric Vehicle Technology Explained*. 2^a ed. Wiley, 2012.
- [5] Rajesh Rajamani. *Vehicle Dynamics and Control*. 2^a ed. New York: Springer, 2011. DOI: 10.1007/978-1-4614-1433-9.